
What Counts as a Contribution

Task W13: The Hack-a-thon

The Hack-a-thon rests on one simple expectation: take part in good faith and contribute to the effort. This document explains what that means in practice, so that everyone walks in on Friday with the same understanding of what a contribution is, what is not enough by itself, and what it looks like to be there with the team.

1 Why This Matters

This project is bigger than a single assignment. It started with students who wanted to leave something behind that would outlast their own courses, and it continues because each new group picks it up and carries it further. The senior developers who will be working alongside you were junior developers last time. They want you there because they want to teach you, and because the project needs people who will carry it forward after they graduate.

Everything built this weekend becomes part of a real game, in a public repository, with your name in its history. It is a lasting record of what CIS students can build together. If you want to be part of something bigger, the Hack-a-thon is where that starts.

2 What a Contribution Is

A contribution is a real artifact: something you made that now exists in the project, or that the team can see, use, react to, or build on. It does not have to be large, polished, or finished. It does have to be something you produced, not something you only watched or approved.

There is room for every skill set. The project has many parts, and any of them is a legitimate place to work. Table 1 lists some of them with examples.

Table 1: Areas of the project and example contributions

Area	Example contributions
Core game	A new mechanic, a fix to existing gameplay code, or a prototype of a feature in its own branch.
Components and systems	A class or subsystem such as combat, inventory, player state, or saving; pseudocode or a UML diagram for one; a data structure for game state.
Database	A table design, a schema change, queries the game needs, or test data.
Website	A new page, a layout, content for the project site, or a fix for something that is broken.
Artwork and assets	Sprites, tiles, icons, interface sketches, or placeholder art the team can replace later.
Documentation	A README section, setup instructions for new contributors, or a written summary of a design decision.
Testing and quality	Test cases, a bug report with steps to reproduce it, or verifying that someone else's fix actually works.
Project and repository	Issues in the tracker, branch cleanup, or build and tooling setup that makes everyone else faster.

If what you want to do is not on this list, it may still count. The test is simple: does it produce something that moves the project forward? If you are unsure, ask a project manager or a senior

developer. A contribution that the team revises, or decides not to use, is still a contribution. That is a normal part of open-source work. The same definition also applies to the Contribution Bonus for work done outside the Hack-a-thon weekend.

If you are working on something real, you are meeting the expectation. You do not need to wonder whether it is big enough or impressive enough.

3 What Does Not Count on Its Own

This section is not meant to make anyone nervous. It is aimed at one narrow thing: being present without producing anything. If you are making something, you are nowhere near this line.

The following, on their own, do not meet the expectation:

- Agreeing with others, such as “I agree,” “good idea,” or “looks good.”
- Reacting to messages or posts without adding anything to them.
- Watching a screen share or sitting in a voice channel without working on anything.
- Restating or summarizing what someone else already did.
- Committing AI-generated code that you cannot explain.

None of these, apart from the last, are bad things to do. Encouragement matters, and a team that cheers each other on works better. The point is that they belong alongside a contribution, not in place of one.

3.1 Reaction versus review

There is an important difference between reacting to someone’s work and reviewing it. Reviewing counts. A review is specific enough that the other person can act on it: you ran their code and found a bug, you tested their feature and wrote down what broke and how, you read their design and proposed a concrete change, or you pointed to a line of code and explained what should change and why.

A useful test: if your feedback could be pasted under anyone’s work and still make sense, it is a reaction, not a review. Table 2 shows the difference.

Table 2: Reaction and review compared

Does not count on its own	Counts
“Nice work on the inventory.”	“The inventory accepts a 21st item when the limit is 20. Here are the steps to reproduce it.”
“I agree with this design.”	“This design works, but saving the whole map as one file will get slow. Here is a proposal to split it by region.”
Reacting to a merged pull request.	Reviewing a pull request and commenting on specific lines before it is merged.

4 Not Knowing How Is Part of It

You are not expected to already know how to do the work. Learning something new over the weekend is one of the main reasons this event exists. If you do not know how to do something yet, that is fine, as long as you are actively working to learn it and you turn that effort into something real.

In practice, that looks like this:

- Pick something, even if it is small, and start.
- Ask a senior developer or project manager to point you in a direction, or to pair with you for a while.
- Use documentation, tutorials, AI tools, and the tools the team is already using to figure out the next step.
- Produce something from the attempt: a draft, a prototype in a branch, a partial solution with notes on where you got stuck, or a written explanation of what you learned that the next person can use.

An honest attempt that produces an artifact is a contribution, even if it is not finished and even if it does not end up in the final game. Waiting for someone to hand you a task, or deciding you cannot help because you do not know enough yet, is not.

4.1 Using AI tools

AI tools are allowed during the Hack-a-thon as a way to learn: to explain a concept, interpret an error message, find syntax, or help you understand code someone else wrote.

Vibe coding has its place too. *Vibe coding* means prompting an AI tool and accepting what it produces without closely reading or understanding the code. It is a great way to explore an idea, build a quick prototype of something you might want to contribute, or grow a personal hobby project and your own skills. Feel free to use it that way. A vibe-coded prototype shared with the team as a proposal, such as a rough version of a screen or mechanic posted in Discord for discussion, counts as a contribution. That is exactly the right way to use it.

What is not acceptable is treating vibe-coded output as production code: committing it to the project without reading it, understanding it, or checking that it fits. The result is often called *AI slop*. It may look finished, but it is often bloated, subtly wrong, or impossible for its submitter to maintain. A prototype shows what you want to build. The code that goes into the project has to be code you understand.

This is not just a course rule. Open-source projects across the industry are pushing back hard against pull requests full of AI slop. In 2026, the [Godot](#) game engine began requiring that all code be human authored, limiting AI to menial tasks such as code completion, and vibe-coded submissions lead to a ban from its repository. [OpenJDK](#), the reference implementation of Java, adopted an interim policy barring AI-generated content from contributions, while still allowing contributors to use AI privately to understand and debug code. Our project is more permissive than either, but the principle is the same: the person submitting the code is responsible for it.

AI-generated code that is dropped into the project without real understanding will be critiqued

closely, and it may be dismissed as a contribution if it is clear that you do not understand it or cannot explain it. Before you commit anything, make sure you could walk a teammate through it and explain why it works. The skill that matters, in this project and in industry, is being able to read code, judge whether it is good, and fix it when it is not. Use AI to build that skill, not to skip it.

5 Being There

The Hack-a-thon begins Friday, November 13, at 1:00 pm with a meeting to get everyone organized. Everyone should attend the opening meeting: in person in Russell Hall 104 if you are in the face-to-face section, or in Discord if you are an online-only student.

Come with at least an idea of what you might want to work on, such as an area of the project from Table 1 that interests you or a skill you want to build. It does not need to be a finished plan. The opening meeting exists to help everyone settle on something, and the senior developers and project managers will help you turn your idea into a place to start. From there, the team splits up and gets to work.

The event continues into Saturday, November 14. There is no official ending time on Saturday, but it will likely wrap up sometime in the afternoon. Do not count on Saturday evening to get your participation in. If the event ends before you have taken part, you will have missed the requirements for this task. Whenever you step away, in the lab or in Discord, let your team know what you were working on and where you left it.

5.1 Online-only students

You are not expected to come to the lab, but you are always welcome to join us in person in Russell Hall 104 for any part of the weekend. Otherwise, you will participate in Discord for the whole event, on the Cyberdemons Roguelike server (<https://discord.gg/QADwedtr2E>). The same expectation applies to you as to everyone else: be there with the team, and contribute artifacts, not just presence.

5.2 In-person students

You are expected to spend part of the weekend in the lab with the team. There is no fixed number of hours and no sign-in sheet. You can come and go, and it is understood that some of you work or have other obligations. As a rough guide, plan to spend at least a few hours in the lab, across more than one visit, rather than a single brief appearance.

The rest of your time can be in Discord. Working online counts, as long as you are actually working: building, testing, writing, or reviewing. Being logged in and listening is not the same as participating.

Keep in mind who is waiting for you in the lab. Everyone on this project wants everyone else there, and this semester that matters more than usual. The senior developers and your fellow in-person students are setting aside their own weekend because they want you there with them. They want to work side by side with you, get to know you, and show you how the project works, so that junior developers can take over parts of the project and keep it going. That kind of teaching happens best in the same room, looking at the same screen. Every hour you spend in the lab is an hour someone can show you how something works, and an hour you might show someone else. Much of that is lost over Discord. When you choose to work online instead of coming in, you are not the only one

who misses out. They do too.

5.3 How participation is judged

After the event, the project managers and I will discuss each person's participation together. We will be looking at two things: whether you were there with the team, in whatever form fits your section, and whether you produced real contributions. A student with real obligations who shows up when they can, and does real work while there, is meeting the expectation.

Your commits and what you post in Discord, together with what we see during the event, are the main evidence of your participation. The Hack-a-thon Contribution Form helps document it.

5.4 If you cannot attend

If you have a genuine conflict with the Hack-a-thon, talk to me personally by Monday, November 2, and bring documentation showing why taking part during the event, either in person or online, is not possible. The Hack-a-thon dates were announced well in advance, so ordinary scheduling choices, such as finishing work for another course or planning to travel home for the weekend, do not qualify. A documented conflict, such as attending a university-sponsored academic conference, is the kind of situation we can plan around. When that is the case, we will work out a reasonable way for you to take part.

Be aware that participation is much easier to confirm when you are there. During the event, the project managers and I see your work as it happens, and that is the clearest evidence there is. Work done after the event to make up for missing it receives much closer scrutiny, because we did not see it develop and need to confirm that it is real, that it is yours, and that it is substantial. Missing the Hack-a-thon is not made up with a quick, simple task afterward. If you do need an alternative, expect it to take real effort.

Come ready to build something, learn something, and help someone. That is all this asks, and it is what the whole team is hoping for.

6 Quick Reference

Table 3: Hack-a-thon contribution expectations at a glance

Item	Detail
Opening meeting	Friday, November 13, at 1:00 pm, in Russell Hall 104 or Discord. Come with an idea of what to work on.
Event window	Into Saturday, November 14, likely ending in the afternoon. If it ends before you take part, you have missed the requirement.
Online-only students	Participate in Discord. You are welcome, but not expected, in the lab.
In-person students	At least a few hours in the lab, across more than one visit. Remaining time can be in Discord, as long as you are working.
What counts	A real artifact you produced: code, design, art, documentation, tests, bug reports, or specific review that others can act on.
What does not count alone	Agreeing, reacting, listening, or watching without producing anything.
Not knowing how	That is okay, as long as you learn and produce something from the attempt.
AI tools	Allowed for learning and prototyping. Committed code you cannot explain may be dismissed.
Cannot attend	Talk to me personally by Monday, November 2, with documentation.